

程序的链接

一个大的程序往往会分成多个源程序文件来编写，因而需要对各个不同源程序文件分别进行编译或汇编，以生成多个不同的目标代码文件，这些目标代码文件中包含指令、数据和其他说明信息。此外，在程序中还会调用一些标准库函数。为了生成一个可执行文件，需要将所有关联到的目标代码文件，包括用到的标准库函数目标文件，按照某种形式组合在一起，形成一个具有统一地址空间的可被加载到存储器直接执行的程序。这种将一个程序的所有关联模块对应的目标代码文件结合在一起，以形成一个可执行文件的过程称为链接。在早期计算机系统中，链接是手动完成的，而现在则由专门的链接程序（linker，也称为链接器）来实现。

了解链接器的工作原理和可执行文件的存储器映像，将有助于养成良好的程序设计习惯，增强程序调试能力，并能够深入理解进程的虚拟地址空间概念。本章主要内容包括：静态链接的概念、目标文件格式、符号及符号表、符号解析、使用静态库链接、重定位信息及重定位过程、可执行文件的存储器映像、可执行文件的加载和共享库动态链接等。

4.1 编译、汇编和静态链接

在第 1 章和第 3 章中都提到过，将高级语言源程序文件转换为可执行目标文件通常分为预处理、编译、汇编和链接四个步骤。前面三个步骤用来对每个模块（即源程序文件）生成可重定位目标文件（relocatable object file）。GCC 生成的可重定位目标文件为 .o 后缀，VC 输出的可重定位目标文件为 .obj 后缀。最后一步用来将若干可重定位目标文件（可能包括若干标准库函数目标模块）组合起来，生成一个可执行目标文件（executable object file）。本书有时将可重定位目标文件和可执行目标文件分别简称可重定位文件和可执行文件。

4.1.1 编译和汇编

下面以 GCC 处理 C 语言程序为例来说明处理过程。可以通过 -v 选项查看 GCC 每一步的处理结果。如果想得到每个处理过程的结果，则可以分别使用 -E、-S 和 -c 选项来进行预处理、编译和汇编，对应的处理工具分别为 cpp、ccl 和 as，处理后得到的文件的文件名后缀分别是 .i、.s 和 .o。

1. 预处理

预处理是从源程序变成可执行程序的第一步，C 预处理程序为 cpp（即 C Preprocessor），主

要用于 C 语言编译器对各种预处理命令进行处理, 包括对头文件的包含、宏定义的扩展、条件编译的选择等, 例如, 对于 `#include` 指示的处理结果, 就是将相应 .h 文件的内容插入到源程序文件中。

GCC 中的预处理命令是 “`gcc -E`” 或 “`cpp`”, 例如, 可用命令 “`gcc -E main.c -o main.i`” 或 “`cpp main.c -o main.i`” 将 `main.c` 转换为预处理后的文件 `main.i`。预处理后的文件是可显示的文本文件。

2. 编译

C 编译器在进行具体的程序翻译之前, 会先对源程序进行词法分析和语法分析, 然后根据分析的结果进行代码优化和存储分配, 最终把 C 语言源程序翻译成汇编语言程序。编译器通常采用对源程序进行多次扫描的方式进行处理, 每次扫描集中完成一项或几项任务, 也可以将一项任务分散到几次扫描去完成。例如, 可以按照以下四趟扫描进行处理: 第一趟扫描进行词法分析; 第二趟扫描进行语法分析; 第三趟扫描进行代码优化和存储分配; 第四趟扫描生成代码。

编译器可以直接产生机器语言代码, 也可以先产生汇编语言代码, 然后再通过汇编程序将汇编语言代码转换为机器语言代码。

GCC 中的编译命令是 “`gcc -S`” 或 “`ccl`”, 例如, 可使用命令 “`gcc -S main.i -o main.s`” 或 “`ccl main.i -o main.s`” 对 `main.i` 进行编译并生成汇编代码文件 `main.s`, 也可以使用命令 “`gcc -S main.c -o main.s`” 或 “`gcc -S main.c`” 直接对 `main.c` 预处理并编译生成汇编代码文件 `main.s`。

3. 汇编

汇编的功能是将编译生成的汇编语言代码转换为机器语言代码。因为通常最终的可执行目标文件由多个不同模块对应的机器语言目标代码组合而形成, 所以, 在生成单个模块的机器语言目标代码时, 不可能确定每条指令或每个数据最终的地址, 也即, 单个模块的机器语言目标代码需要重新定位, 因此, 通常把汇编生成的机器语言目标代码文件称为可重定位目标文件。

GCC 中的汇编命令是 “`gcc -c`” 或 “`as`”。例如, 可用命令 “`gcc -c main.s -o main.o`” 或 “`as main.s -o main.o`” 对汇编语言代码文件 `main.s` 进行汇编, 以生成可重定位目标文件 `main.o`。也可以使用命令 “`gcc -c main.c -o main.o`” 或 “`gcc -c main.c`” 直接对 `main.c` 进行预处理并编译生成可重定位目标文件 `main.o`。

4.1.2 可执行目标文件的生成

链接的功能是将所有关联的可重定位目标文件组合起来, 以生成一个可执行文件。例如, 对于图 4.1 所示的两个模块 `main.c` 和 `test.c`, 假定通过预处理、编译和汇编, 分别生成了可重定

<pre> 1 int add(int, int); 2 int main() 3 { 4 return add(20, 13); 5 }</pre>	<pre> 1 int add(int i, int j) 2 { 3 int x=i+j; 4 return x; 5 }</pre>
a) main.c文件	b) test.c文件

图 4.1 两个源程序文件组合成一个可执行目标文件示例

位目标文件 `main.o` 和 `test.o`，则可以用命令 “`gcc -o test main.o test.o`” 或 “`ld -o test main.o test.o`” 来生成可执行文件 `test`。这里，`ld` 是静态链接器命令。

当然，也可以用一个命令 “`gcc -o test main.c test.c`” 来实现对源程序文件 `main.c` 和 `test.c` 的预处理、编译和汇编，并将两个可重定位目标文件 `main.o` 和 `test.o` 进行链接，最终生成可执行目标文件 `test`。命令 “`gcc -o test main.c test.c`” 的功能如图 4.2 所示。

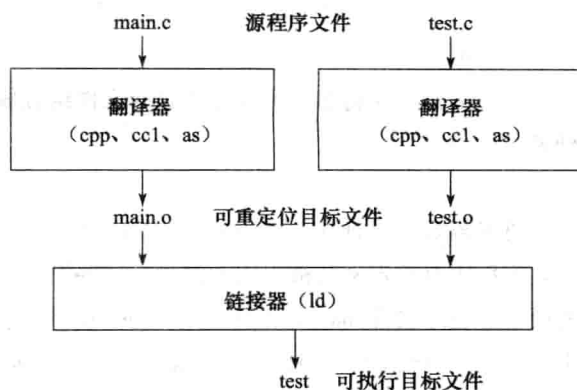


图 4.2 可执行目标文件 `test` 的生成过程

可重定位目标文件和可执行目标文件都是机器语言目标文件，所不同的是前者是单个模块生成的，而后者是多个模块组合而成的。因而，对于前者，代码总是从 0 开始，而对于后者，代码在操作系统规定的虚拟地址空间中产生。有关虚拟地址空间的概念在第 6 章介绍。

例如，通过 “`objdump -d test.o`” 命令显示的可重定位目标文件 `test.o` 的结果如下。

```

00000000 <add>:
   0: 55          push    %ebp
   1: 89 e5       mov     %esp, %ebp
   3: 83 ec 10    sub     $0x10, %esp
   6: 8b 45 0c    mov     0xc(%ebp), %eax
   9: 8b 55 08    mov     0x8(%ebp), %edx
  c: 8d 04 02    lea     (%edx,%eax,1), %eax
  f: 89 45 fc    mov     %eax, -0x4(%ebp)
12: 8b 45 fc    mov     -0x4(%ebp), %eax
15: c9         leave  %eax
16: c3         ret
  
```

通过 “`objdump -d test`” 命令显示的可执行目标文件 `test` 的结果如下。

```

080483d4 <add>:
080483d4: 55          push    %ebp
080483d5: 89 e5       mov     %esp, %ebp
080483d7: 83 ec 10    sub     $0x10, %esp
080483da: 8b 45 0c    mov     0xc(%ebp), %eax
080483dd: 8b 55 08    mov     0x8(%ebp), %edx
080483e0: 8d 04 02    lea     (%edx,%eax,1), %eax
080483e3: 89 45 fc    mov     %eax, -0x4(%ebp)
080483e6: 8b 45 fc    mov     -0x4(%ebp), %eax
080483e9: c9         leave  %eax
080483ea: c3         ret
  
```

上述给出的通过 `objdump` 命令输出的结果包括指令的地址、指令机器代码和反汇编出来的汇编指令代码。可以看出，在可重定位目标文件 `test.o` 中 `add` 函数的起始地址为 0；而在可执行目标文件 `test` 中 `add` 函数的起始地址为 `0x080483d4`。

实际上，可重定位目标文件和可执行目标文件都不是可以直接显示的文本文件，而是不可

显示的二进制文件，它们都按照一定的格式以二进制字节序列构成一种目标文件，其中包含二进制代码区、只读数据区、已初始化数据区和未初始化数据区等，每个信息区称为一个节（section），如代码节（.text）、只读数据节（.rodata）、已初始化全局数据节（.data）和未初始化全局数据节（.bss）等。

静态链接器在将多个可重定位目标文件组合成一个可执行目标文件时，主要完成以下两个任务。

（1）符号解析

符号解析（symbol resolution）的目的就是将每个符号的引用与一个确定的符号定义建立关联。程序中有被定义和被引用的符号，这些符号包括变量名和函数名。例如，对于图 4.1 所示的两个源程序文件 main.c 和 test.c，在 main.c 中定义了符号 main，并引用了符号 add；在 test.c 中则定义了符号 add。因此，在链接时，需要将 main.o 中引用的符号 add 和 test.o 中定义的符号 add 建立关联。对于语句“int *xp = &x;”，则引用了符号 x 并对符号 xp 进行了定义。编译器将所有符号存放在可重定位目标文件的符号表（symbol table）中。符号表是一个结构数组，每个表项包含符号名、长度和位置信息等。

（2）重定位

可重定位目标文件中的代码区和数据区都是从地址 0 开始的，链接器需要将不同模块中相同的节合并起来生成一个新的单独的节，并将合并后的代码区和数据区按照操作系统确定的虚拟地址空间划分（也称存储器映像）来重新确定位置。例如，对于 32 位 Linux 系统存储器映像，其代码区总是从地址 0x08048000 开始，而数据区总是在代码区后面的第一个 4KB 对齐的地址处开始。因而链接器需要重新确定每条指令和每个数据的地址，并且在指令中需要明确给定所引用符号的地址，这种重新确定代码和数据的地址并更新指令中被引用符号地址的工作称为重定位（relocation）。

使用链接的第一个好处就是“模块化”，它能使一个程序被划分成多个模块，由不同的程序员进行编写，并且可以构建公共的函数库（如数学函数库、标准 I/O 函数库等）以提供给不同的程序进行重用。采用链接的第二个好处是“效率高”，每个模块可以分开编译，在程序修改时只需重新编译那些修改过的源程序文件，然后再重新链接，因而从时间上来说能够提高程序开发的效率；同时，因为源程序文件中无需包含共享库的所有代码，只要直接调用即可，而且在可执行文件运行时的内存中，也只需要包含所调用函数的代码而不需要包含整个共享库，所以链接也有效地提高了空间利用率。

4.2 目标文件格式

目标代码（object code）指编译器或汇编器处理源代码后所生成的机器语言目标代码。目标文件（object file）指存放目标代码的文件。

4.2.1 ELF 目标文件格式

目标文件中包含可直接被 CPU 执行的机器代码以及代码在运行时使用的数据，还有其他

的如重定位信息和调试信息等。不过，目标文件中唯一的要素是机器代码及其使用的数据，例如，用于嵌入式系统的目标文件可能仅仅含有机器代码及其使用数据。

目标文件格式有许多不同的种类。最初，不同的计算机都拥有自身独特的格式，但随着 UNIX 和其他可移植操作系统的问世，人们定义了一些标准目标文件格式，并在不同的系统上使用它们。最简单的目标文件格式是 DOS 操作系统的 COM 文件格式，它是一种仅由代码和数据组成的文件，而且始终被加载到某个固定位置。其他的目标文件格式（如 COFF 和 ELF）都比较复杂，由一组严格定义的数据结构序列组成，这些复杂格式的规范说明书一般会有许多页。System V UNIX 的早期版本使用的是通用目标文件格式（Common Object File Format，简称 COFF）。Windows 使用的是 COFF 的一个变种，称为可移植可执行格式（Portable Executable，简称 PE）。现代 UNIX 操作系统，如 Linux、BSD UNIX 等，主要使用可执行可链接格式（Executable and Linkable Format，简称 ELF），本章采用 ELF 标准二进制文件格式进行说明。

目标文件既可用于程序的链接，也可用于程序的执行。图 4.3 说明了 ELF 目标文件格式的基本框架。图 4.3a 是链接视图，主要由不同的节（section）组成，节是 ELF 文件中具有相同特征的最小可处理信息单位，不同的节描述了目标文件中不同类型的信息及其特征，例如，代码节（.text）、只读数据节（.rodata）、已初始化全局数据节（.data）、未初始化全局数据节（.bss）等。图 4.3b 是执行视图，主要由不同的段（segment）组成，描述了目标文件中的节如何映射到存储空间的段中。可以多个节映射到同一个段，例如，可以合并节 .data 和节 .bss 的内容，并映射到一个可读可写数据段中。

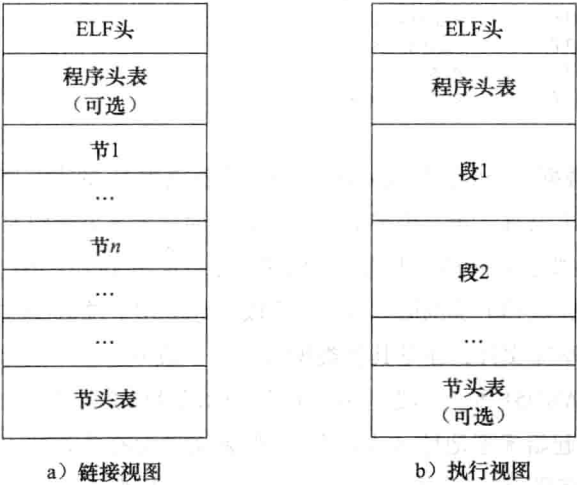


图 4.3 ELF 目标文件的两种视图

前面提到通过预处理、编译和汇编三个步骤后，可生成可重定位目标文件，多个关联的可重定位目标文件经过链接后生成可执行目标文件。这两类目标文件对应的 ELF 视图不同，显然，可重定位目标文件对应链接视图，而可执行目标文件对应执行视图。

程序头表用来指示系统如何创建进程的存储器映像。用于创建进程存储器映像的可执行文件和共享库文件必须具有程序头表，而可重定位目标文件无需程序头表。节头表包含文件中各节的说明信息，每个节在该表中都有一个与之对应的项，每一项都指定了节名和节大小之类的

信息。用于链接的目标文件必须具有节头表，例如，可重定位目标文件就一定要有节头表。

4.2.2 可重定位目标文件格式

可重定位目标文件主要包含代码部分和数据部分，它可以与其他可重定位目标文件链接，从而创建可执行目标文件、共享库文件或其他可重定位目标文件。如图 4.4 所示，ELF 可重定位目标文件由 ELF 头、节头表以及夹在 ELF 头和节头表之间的各个不同的节组成。

1. ELF 头

ELF 头位于目标文件的起始位置，包含文件结构说明信息。ELF 头的数据结构分 32 位系统对应结构和 64 位系统对应结构，以下是 32 位系统对应的数据结构。

```
#define EI_NIDENT          16
typedef struct {
    unsigned char    e_ident[EI_NIDENT];
    Elf32_Half      e_type;
    Elf32_Half      e_machine;
    Elf32_Word      e_version;
    Elf32_Addr      e_entry;
    Elf32_Off       e_phoff;
    Elf32_Off       e_shoff;
    Elf32_Word      e_flags;
    Elf32_Half      e_ehsize;
    Elf32_Half      e_phentsize;
    Elf32_Half      e_phnum;
    Elf32_Half      e_shentsize;
    Elf32_Half      e_shnum;
    Elf32_Half      e_shstrndx;
} Elf32_Ehdr;
```

ELF 头
.text 节
.rodata 节
.data 节
.bss 节
.symtab 节
.rel.text 节
.rel.data 节
.debug 节
.line 节
.strtab 节
节头表

图 4.4 ELF 可重定位目标文件

在 32 位 ELF 头的数据结构中，字段 `e_ident` 是一个长度为 16 的字节序列，其中，最开始的 4 字节用来标识是否为 ELF 文件，第一个字节为 0x7F，后面三个字节分别为 ‘E’、‘L’、‘F’。再后面的 12 个字节中，主要包含一些标识信息，例如，标识是 32 位还是 64 位格式、标识数据按小端还是大端方式存放、标识 ELF 头的版本号等。字段 `e_type` 用于说明目标文件的类型是可重定位文件、可执行文件、共享库文件，还是其他类型文件。字段 `e_machine` 用于指定机器结构类型，如 IA-32、SPARC V9、AMD64 等。字段 `e_version` 用于标识目标文件版本。字段 `e_entry` 用于指定系统将控制权转移到的起始虚拟地址（入口点），如果文件没有关联的入口点，则为零。例如，对于可重定位文件，此字段为 0。字段 `e_ehsize` 用于说明 ELF 头的大小（以字节为单位）。字段 `e_shoff` 指出节头表在文件中的偏移量（以字节为单位）。字段 `e_shentsize` 表示节头表中一个表项的大小（以字节为单位），所有表项大小相同。字段 `e_shnum` 表示节头表中的项数。因此 `e_shentsize` 和 `e_shnum` 共同指定了节头表的大小（以字节为单位）。仅 ELF 头在文件中具有固定位置，即总是在最开始的位置，其他部分的位置由 ELF 头和节头表指出，不需要具有固定的顺序。

2. 节头表

节头表由若干个表项组成，每个表项描述相应的一个节的节名、位置和长度等信息，目标文件中的每个节都有一个表项与之对应。

3. 节

节是 ELF 文件中的主体信息，包含了链接过程所用的目标代码信息，包括指令、数据、符号表和重定位信息等。一个典型的 ELF 可重定位目标文件中包含下面几个节。

.text：目标代码部分。

.rodata：只读数据，如 printf 语句中的格式串、开关语句（如 switch-case）的跳转表等。

.data：已初始化的全局变量。

.bss：未初始化的全局变量。因为未初始化变量没有具体的值，所以无需在目标文件中分配用于保存值的空间，也即它在目标文件中不占据实际的磁盘空间，仅仅是一个占位符。目标文件中区分已初始化和未初始化全局变量是为了提高空间利用率。

对于 auto 型局部变量，因为它们在运行时被分配在栈中，因此既不出现在 .data 节，也不出现在 .bss 节。

.symtab：符号表（symbol table）。在程序中被定义和引用的函数名和全局变量名都属于符号，与这些符号相关的信息被保存在符号表中。每个可重定位目标文件都有一个 .symtab 节。

.rel.text：.text 节相关的可重定位信息。当链接器将某个目标文件和其他目标文件组合时，.text 节中的代码被合并后，一些指令中引用的操作数地址信息或跳转目标指令位置信息等都可能要被修改。通常，调用外部函数或者引用全局变量的指令中的地址字段需要修改。

.rel.data：.data 节相关的可重定位信息。当链接器将某个目标文件和其他目标文件组合时，.data 节中的代码被合并后，一些全局变量的地址可能被修改。

.debug：调试用符号表，有些表项对程序中定义的局部变量和类型定义进行说明，有些表项对程序中定义和引用的全局变量进行说明。只有带-g 选项的 gcc 命令才会得到这张表。

.line：C 源程序中的行号和 .text 节中机器指令之间的映射。只有带-g 选项的 gcc 命令才会得到这张表。

.strtab：字符串表，包括 .symtab 节和 .debug 节中的符号以及节头表中的节名。字符串表就是以 null 结尾的字符串序列。

4.2.3 可执行目标文件格式

可执行目标文件由链接器将若干个相互关联的可重定位目标文件组合起来而生成，可重定位文件中的代码和数据的地址是相对于起始地址 0 而得到的，而可执行文件中代码和数据的地址则按照操作系统规定的存储器映像来确定起始地址，并且可重定位文件中代码和数据的地址将会被修改，使得它们被重定位到运行时的虚拟存储空间中相应地址处。

ELF 可执行目标文件由 ELF 头、程序头表、节头表以及夹在程序头表和节头表之间的各个不同的节组成，如图 4.5 所示。可执行目标文件格式与可重定位目标文件格式类似，例如，这两种格式中 ELF 头的数据结构一样，

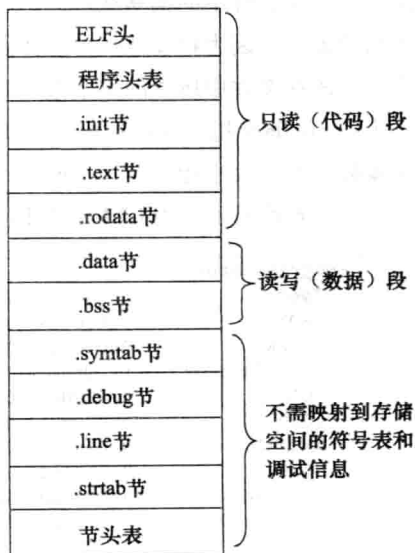


图 4.5 ELF 可执行目标文件

.text 节、.rodata 节和 .data 节中除了有些重定位地址不同以外，大部分都类似。与 ELF 可重定位目标文件格式相比，ELF 可执行目标文件的不同点主要有：

① ELF 头中字段 `e_entry` 给出系统将控制权转移到的起始的虚拟地址（入口点），即执行程序时第一条指令的地址。而在可重定位目标文件中，此字段为 0。

② 多了一个 .init 节，其中定义了一个 `_init` 函数，用于可执行目标文件开始执行时的初始化工作。

③ 少了两个 .rel 节，因为可执行目标文件中的指令和数据已被重定位，故可去掉用于重定位的节。

④ 多了一个 程序头表，也称段头表（segment header table），它是一个结构数组。可执行目标文件中所有代码的位置连续，所有只读数据的位置连续，所有可读可写数据的位置连续。因而，这些连续的片段（chunk）被映射到存储空间（实际上就是虚拟地址空间）中的一个存储段，程序头表用于描述这种映射关系，一个表项说明一个连续的片段或一个特殊的节。由 ELF 头中的字段 `e_phentsize` 和 `e_phnum` 分别指定程序头表表项大小和表项数。

32 位系统的程序头表中每个表项具有以下数据结构：

```
typedef struct {
    Elf32_Word    p_type;
    Elf32_Off     p_offset;
    Elf32_Addr    p_vaddr;
    Elf32_Addr    p_paddr;
    Elf32_Word    p_filesz;
    Elf32_Word    p_memsz;
    Elf32_Word    p_flags;
    Elf32_Word    p_align;
} Elf32_Phdr;
```

`p_type` 描述存储段的类型或特殊节的类型。例如，是否为可装入段（`PT_LOAD`），是否是特殊的动态节（`PT_DYNAMIC`），是否是特殊的解释程序节（`PT_INTERP`）。`p_offset` 指出本段的首字节在文件中的偏移地址。`p_vaddr` 指出本段首字节的虚拟地址。`p_paddr` 指出本段首字节的物理地址，因为物理地址由操作系统根据情况动态确定，因而该信息通常是无效的。`p_filesz` 指出本段在文件中所占的字节数，可以为 0。`p_memsz` 指出本段在存储器中所占字节数，也可以为 0。`p_flags` 指出存取权限。`p_align` 指出对齐方式，用一个模数表示，为 2 的正整数幂，通常模数与页面大小相关，若页面大小为 4KB，则模数为 2^{12} 。

图 4.6 给出了使用 GNU `readelf` 显示的某个可执行目标文件的程序头表信息。

Program Headers:							
Type	offset	VirtAddr	PhysAddr	FileSiz	MemSiz	Flg	Align
PHDY	0x000034	0x08048034	0x08048034	0x00100	0x00100	R E	0x4
INTERP	0x000134	0x08048134	0x08048134	0x00013	0x00013	R	0x1
[Requesting program interpreter: /lib/ld-linux.so.2]							
LOAD	0x000000	0x08048000	0x08048000	0x004d4	0x004d4	R E	0x1000
LOAD	0x000f0c	0x08049f0c	0x08049f0c	0x00108	0x00110	RW	0x1000
DYNAMIC	0x000f20	0x08049f20	0x08049f20	0x000d0	0x000d0	RW	0x4
NOTE	0x000148	0x08048148	0x08048148	0x00044	0x00044	R	0x4
GNU_STACK	0x000000	0x00000000	0x00000000	0x00000	0x00000	RW	0x4
GNU_RELRO	0x000f0c	0x08049f0c	0x08049f0c	0x000f4	0x000f4	R	0x1

图 4.6 某可执行文件的程序头表中部分信息

图 4.6 给出的程序头表中有 8 个表项，其中有两个是可装入段（Type = LOAD）对应的表项信息。第一个可装入段对应可执行目标文件中第 0x00000 ~ 0x004d3 字节的内容（包括 ELF 头、程序头表以及 .init、.text 和 .rodata 节），被映射到虚拟地址 0x8048000 开始的长度为 0x004d4 字节的区域，按 $0x1000 = 2^{12} = 4\text{KB}$ 对齐，具有只读/执行权限（Flg = RE），因此它是一个只读代码段（read-only code segment）。第二个可装入段对应可执行目标文件中第 0x000f0c 开始的长度为 0x00108 字节的内容（即 .data 节），被映射到虚拟地址 0x8049f0c 开始的长度为 0x00110 字节的存储区域，在 0x00110 = 272 字节的存储区中，前 0x00108 = 264 字节用 .data 节的内容来初始化，而后面的 $272 - 264 = 8$ 个字节对应 .bss 节，被初始化为 0，该段按 $0x1000 = 4\text{KB}$ 对齐，具有可读可写权限（Flg = RW），因此，它是一个可读写数据段（read/write data segment）。

从这个例子可以看出，.data 节在可执行目标文件中占用了相应的磁盘空间，在存储器中也需要给它分配相同大小的空间；而 .bss 节在文件中不占用磁盘空间，但在存储器中需要给它分配相应大小的空间。

4.3 符号表和符号解析

4.3.1 符号和符号表

链接器在生成一个可执行目标文件时，必须完成符号解析，而要进行符号解析，则需要用到符号表。通常目标文件中都有一个符号表，表中包含了在程序模块中被定义和引用的所有符号的相关信息。对于某个 C 程序模块 m 来说，包含在符号表中的符号有以下三种不同类型。

① 在模块 m 中定义并被其他模块引用的全局符号（global symbol）。这类符号包括非静态的函数名和被定义为不带 static 属性的全局变量名。

② 由其他模块定义并被 m 引用的外部符号（external symbol），包括在其他模块定义的外部函数名和外部变量名。

③ 在模块 m 中定义并在 m 中引用的本地符号（local symbol）。这类符号包括带 static 属性的函数名和全局变量名。这类在一个过程（函数）内部定义的带 static 属性的本地变量不在栈中管理，而是被分配在静态数据区，即编译器为它们在节 .data 或 .bss 中分配空间。如果在模块 m 内有两个函数使用了同名 static 本地变量，则需要为这两个变量都分配空间，并作为两个不同的符号记录到符号表中。

例如，对于以下同一个模块中的两个函数 func1 和 func2，假定它们都定义了 static 本地变量 x 且都被初始化，则编译器在该模块的 .data 节中同时为这两个变量分配空间，并在符号表中构建两个符号 func1.x 和 func2.x 的关联信息。

```

1 int func1()
2 {
3     static int x=0;
4     return x;
5 }
6
7 int func2()
8 {
```

```

9      static int x=1;
10     return x;
11 }

```

注意上述三类符号不包括分配在栈中的非静态局部变量（auto 变量），链接器不需要这类变量的信息，因而它们不包含在由节 .symtab 定义的符号表中。

例如，对于图 4.7 给出的两个源程序文件 main.c 和 swap.c 来说，在 main.c 中的全局符号有 buf 和 main，外部符号有 swap；在 swap.c 中的全局符号有 bufp0 和 swap，外部符号有 buf，本地符号有 bufp1。swap.c 中的 temp 是局部变量，是在运行时动态分配的，它不用记录在符号表中。

```

1 void swap(void);
2
3 int buf[2] = {1, 2};
4
5 int main()
6 {
7     swap();
8     return 0;
9 }

```

a) main.c

```

1 extern int buf[];
2
3 int *bufp0 = &buf[0];
4 static int *bufp1;
5
6 void swap()
7 {
8     int temp;
9     bufp1 = &buf[1];
10    temp = *bufp0;
11    *bufp0 = *bufp1;
12    *bufp1 = temp;
13 }

```

b) swap.c

图 4.7 两个源程序文件模块

ELF 文件中包含的符号表中每个表项具有以下数据结构。

```

typedef struct {
    Elf32_Word      st_name;
    Elf32_Addr      st_value;
    Elf32_Word      st_size;
    unsigned char    st_info;
    unsigned char    st_other;
    Elf32_Half      st_shndx;
} Elf32_Sym;

```

字段 st_name 给出符号在字符串表中的索引（字节偏移量），指向在字符串表（.strtab 节）中的一个以 null 结尾的字符串，即符号。st_value 给出符号的值，在可重定位目标文件中，是指符号所在位置相对于所在节起始位置的字节偏移量。例如，符号 buf 在 .data 节的偏移量为 0；在可执行目标文件和共享目标文件中，则是符号所在的虚拟地址。st_size 给出符号所表示对象的字节个数。例如，若符号是函数名，则是指函数所占字节个数，若符号是变量，则是指变量所占字节个数。如果符号表示的内容没有大小或大小未知，则值为 0。

字段 st_info 指出符号的类型和绑定属性，从以下定义的宏可以看出，符号类型占低 4 位，符号绑定属性占高 4 位。

```

#define ELF32_ST_BIND(info)      ((info) >> 4)
#define ELF32_ST_TYPE(info)     ((info) & 0xf)
#define ELF32_ST_INFO(bind, type) (((bind) << 4) + ((type) & 0xf))

```

符号类型可以是未指定 (NOTYPE)、变量 (OBJECT)、函数 (FUNC)、节 (SECTION) 等。当类型为“节”时, 其表项主要用于重定位。绑定属性可以是本地 (LOCAL)、全局 (GLOBAL)、弱 (WEAK) 等。其中, 本地符号指在包含其定义的目标文件的外部是不可见的, 名称相同的本地符号可存在于多个文件中而不会相互干扰。全局符号对于合并的所有目标文件都可见。弱符号与全局符号类似, 但其定义具有较低的优先级, 详见 4.3.2 节。

字段 `st_other` 指出符号的可见性。通常在可重定位目标文件中指定可见性, 它定义了当符号成为可执行目标文件或共享目标库的一部分后访问该符号的方式。字段 `st_shndx` 用于指出符号所在节在节头表中的索引。有些符号属于三种特殊伪节 (pseudo section) 之一, 伪节在节头表中没有相应的表项, 无法表示其索引值, 因而用以下特殊的索引值表示: ABS 表示该符号不会由于重定位而发生值的改变, 即不应该被重定位; UNDEF 表示未定义符号, 即在本模块引用而在其他模块定义的符号; COMMON 表示还未被分配位置的未初始化的变量。对于 COMMON 类型的符号, 其 `st_value` 字段给出的是对齐要求, 而 `st_size` 给出的是最小长度。

可通过 GNU READELF 工具显示符号表。例如, 对于图 4.7 给出的两个源程序模块文件 `main.c` 和 `swap.c`, 可使用命令 `readelf -s main.o` 查看 `main.o` 中的符号表, 其最后三个表项显示结果如图 4.8 所示。

Num:	Value	Size	Type	Bind	Ot	Ndx	Name
8:	0	8	OBJECT	GLOBAL	0	3	buf
9:	0	17	FUNC	GLOBAL	0	1	main
10:	0	0	NOTYPE	GLOBAL	0	UND	swap

图 4.8 main.o 中部分符号表信息

从显示结果可看出, `main` 模块的三个全局符号中, `buf` 是变量 (Type = OBJECT), 它位于节头表中第三个表项 (Ndx = 3) 对应的 `.data` 节中偏移量为 0 (Value = 0) 处, 占 8 个字节 (Size = 8); `main` 是函数 (Type = FUNC), 它位于节头表中第一个表项对应的 `.text` 节中偏移量为 0 处, 占 17 个字节; `swap` 是未指定 (NOTYPE) 且无定义 (UND) 的符号, 说明 `swap` 是在 `main` 中被引用的由外部模块定义的符号。

使用 GNU READELF 工具显示可重定位目标文件 `swap.o` 符号表中最后四个表项结果如图 4.9 所示。

Num:	Value	Size	Type	Bind	Ot	Ndx	Name
8:	0	4	OBJECT	GLOBAL	0	3	bufp0
9:	0	0	NOTYPE	GLOBAL	0	UND	buf
10:	0	39	FUNC	GLOBAL	0	1	swap
11:	4	4	OBJECT	LOCAL	0	COM	bufp1

图 4.9 swap.o 中部分符号表信息

可以看出, `swap` 模块的四个符号中有三个全局符号和一个本地符号。其中 `bufp0` 是全局变量, 它位于节头表中第三个表项对应的 `.data` 节中偏移量为 0 处, 占 4 个字节; `buf` 是未指定的且无定义的全局符号, 说明 `buf` 是在 `swap` 中被引用的由外部模块定义的符号; `swap` 是函数,

它位于节头表中第一个表项对应的 .text 节中偏移量为 0 处, 占 39 个字节; bufp1 是未分配位置且未初始化 (Ndx = COM) 的本地变量, 按 4 字节边界对齐, 至少占 4 个字节, 当 swap 模块被链接时, bufp1 将作为 .bss 节中的一个变量来分配空间。注意, swap 模块中的变量 temp 是函数内的局部变量, 因而不在于符号表中说明。

4.3.2 符号解析

符号解析的目的是将每个模块中引用的符号与某个目标模块中的定义符号建立关联。每个定义符号在代码段或数据段中都被分配了存储空间, 因此, 将引用符号与对应的定义符号建立关联后, 就可以在重定位时将引用符号的地址重定位为相关联的定义符号的地址。

对于在一个模块中定义且在同一个模块中被引用的本地符号, 链接器的符号解析会比较容易进行, 因为编译器会检查每个模块中的本地符号是否具有唯一的定义, 所以, 只要找到第一个本地定义符号与之关联即可。对于跨模块的全局符号的解析, 则比较困难。

编译器在对源程序编译时, 会把每个全局符号输出到汇编代码文件中, 每个全局符号或者是强符号或者是弱符号。汇编器把全局符号的强、弱特性隐含地编码在可重定位目标文件的符号表中, 以供链接时符号解析所需。

1. 全局符号的强、弱特性

强、弱符号的定义如下: 函数名和已初始化的全局变量名是强符号, 未初始化的全局变量名是弱符号。例如, 在图 4.7 中的两个模块中, main、buf、swap 和 bufp0 是强符号, bufp1 为本地符号, 而本地符号没有强弱之分, temp 则是局部变量, 不包含在符号表中。

链接器根据以下强符号和弱符号的处理规则来处理多重定义符号。

规则 1: 强符号不能多次定义。也即强符号只能被定义一次, 否则链接错误。

规则 2: 若一个符号被说明为一次强符号定义和多次弱符号定义, 则按强符号定义为准。

规则 3: 若有多个弱符号定义, 则任选其中一个。

例如, 对于图 4.10 所示的两个模块 main.c 和 p1.c, 因为强符号 x 被重复定义了两次, 所以链接器将输出一条出错信息。

<pre>int x=10; int p1(void); int main() { x=p1(); return x; }</pre>	<pre>int x=20; int p1() { return x; }</pre>
a) main.c文件	b) p1.c文件

图 4.10 两个强符号定义的例子

考察图 4.11 所示例子中的符号 y 和符号 z 的情况。

图 4.11 中, 符号 y 在 main.c 中是强符号, 在 p1.c 中是弱符号, 根据规则 2 可知, 链接器将 main.o 符号表中的符号 y 作为其唯一定义符号, 而在 p1 模块中的 y 作为引用符号, 其地址等于 main 模块中定义符号 y 的地址, 也即这两个 y 是同一个变量。在 main 函数调用 p1 函数后, y 的值从初始化的 100 被修改为 200, 因而, 在 main 函数中用 printf 打印出来后 y 的值为

200, 而不是 100。

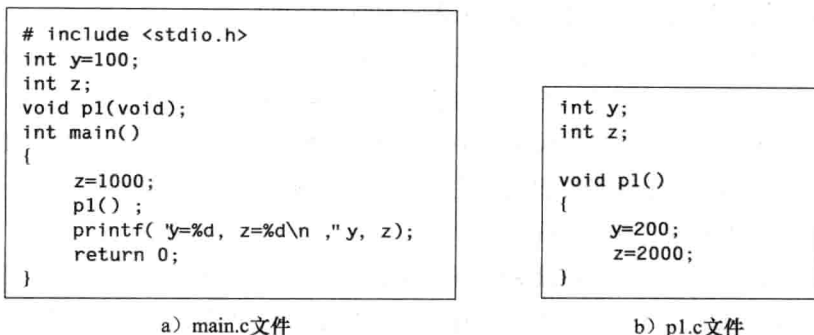


图 4.11 同类型定义符号的例子

对于符号 z , 情况也类似。因为符号 z 在 `main` 和 `p1` 模块都没有初始化, 因此, 在两个模块中都是弱符号, 按照规则 3 可知, 链接器将其中的一个符号作为唯一定义符号, 如果按先后顺序来定, 则链接器将 `main` 模块中定义的符号 z 作为唯一定义符号, 而 `p1` 模块中的 z 作为引用符号, 符号 z 的地址为 `main` 模块中定义的地址。在 `main` 函数调用 `p1` 函数后, z 的值从 1000 被修改为 2000, 因而, 在 `main` 函数中用 `printf` 打印出来后 z 的值为 2000, 而不是 1000。

上述例子说明, 如果在两个不同模块定义相同变量名, 那么很可能会发生程序员意想不到的结果。

特别当两个重复定义的变量具有不同类型时, 更容易出现难以理解的结果。例如, 对于图 4.12 所示的例子, 全局变量 d 在 `main` 模块中为 `int` 型强符号, 在 `p1` 中是 `double` 型弱符号。根据规则 2 可知, 链接器将 `main.o` 符号表中的符号 d 作为其唯一定义符号, 因而其地址和所占字节数等于 `main` 模块中定义符号 d 的地址和字节数, 因此符号长度为 `int` 型大小, 即 4 个字节, 而不是 `double` 型变量的 8 个字节。由于 `p1.c` 中的 d 为引用符号, 因而其地址与 `main` 中变量 d 的地址相同, 在 `main` 函数调用 `p1` 函数后, 地址 $\&d$ 中存放的是 `double` 型浮点数 1.0 对应的低 32 位机器数 00000000H, 地址 $\&x$ 中存放的是 `double` 型浮点数 1.0 对应的高 32 位机器数 3FF00000H, 如图 4.12c 所示。因而, 在 `main` 函数中用 `printf` 打印出来后 d 的值为 0, x 的值是 1 072 693 248。可见 x 的值被 `p1.c` 中的变量 d 给冲掉了。这里, `double` 型浮点数 1.0 对应的机器数为 3FF0 0000 0000 0000H。

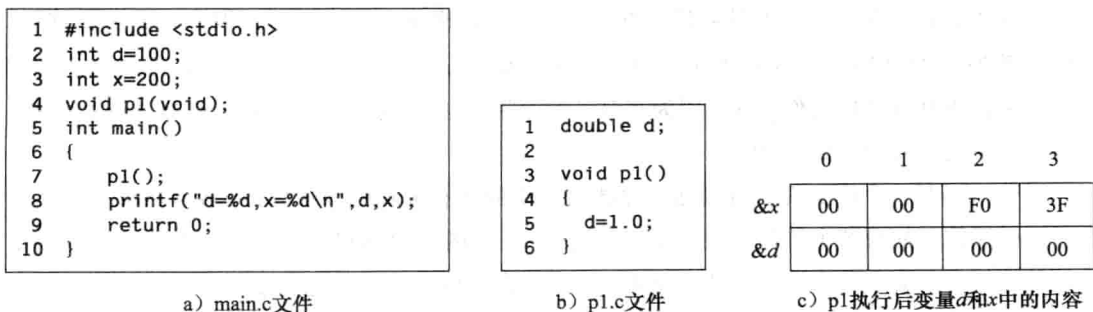


图 4.12 不同类型定义符号例子

对于两个完全相同的结构 (struct) 类型变量, 由不同的编译器分别按照不同的对齐规则进行编译时, 链接生成的程序可能会带来无法预料的结果!

上述由于多重定义变量引起的值的改变往往是在没有任何警告的情况下发生的, 而且通常是在程序执行了一段时间后才表现出来, 并且远离错误发生源, 甚至错误发生源在另一个模块。对于由成百上千个模块组成的大型程序的开发, 这种问题将更加麻烦, 如果没有对变量定义进行规范, 那将很难避免这类错误的发生。可以使用相应的选项命令 (如 `-fno-common`) 来告诉链接器在遇到多重定义符号时输出警告信息。

解决上述问题的办法是, 尽量避免使用全局变量, 一定需要用的话, 就把全局变量定义为 `static`, 这样就没有强弱之分, 而且不会和其他全局符号产生冲突, 如果其他模块需要引用它, 就将它封装成函数。此外, 尽量要给全局变量赋初值使其变成强符号, 而外部全局变量则尽量使用 `extern`。对于程序员来说最好能了解链接器是如何工作的, 如果不了解, 那么就要养成良好的编程习惯。

2. 符号解析过程

编译系统通常会提供一种将多个目标模块打包成一个单独的库文件的机制, 这个库文件就是 静态库 (static library)。在构建可执行文件时只需指定库文件名, 链接器会自动到库中寻找那些应用程序用到的目标模块, 并且只把用到的模块从库中拷贝出来。

程序中的符号包括变量名和函数名, 它们在程序中可能出现在定义处, 称为 符号的定义; 也可能出现在引用处, 称为 符号的引用。为叙述方便起见, 本教材将定义处的符号和引用处的符号分别称为 定义符号 和 引用符号。例如, 对于图 4.12 中的符号 d , 在 `main.c` 第 2 行中是定义符号, 其余地方都是引用符号, 其中, 在 `main.c` 中有一处 (第 8 行) 引用, 在 `p1.c` 中有一处 (第 5 行) 引用。

链接器按照所有可重定位目标文件和静态库文件出现在命令行中的顺序从左至右依次扫描它们, 在此期间它要维护多个集合。其中, 集合 E 是指将被合并到一起组成可执行文件的所有目标文件集合; 集合 U 是未解析符号的集合, 未解析符号是指还未与对应定义符号关联的引用符号; 集合 D 是指当前为止已被加入到 E 的所有目标文件中定义符号的集合。

符号解析开始时, 集合 E 、 U 、 D 中都是空的。然后按照以下过程进行符号解析。

① 对命令行中的每一个输入文件 f , 链接器确定它是目标文件还是库文件, 如果它是目标文件, 就把 f 加入到 E , 根据 f 中未解析符号和定义符号分别对 U 、 D 集合进行修改, 然后处理下一个输入文件。例如, 对于图 4.12 中的符号 d , 在处理 `main.o` 文件时, 因为 d 是定义符号, 所以 d 被加入到 D 中; 而对于 d 的引用, 因为是可以与 d 的定义关联的, 故 d 不被加入到 U 中。然后, 再处理目标文件 `p1.o`, 因为其对 d 的引用可以与 D 中已有的定义符号 d 建立关联, 因此, 也不会将 d 加入到 U 中。

② 如果 f 是一个库文件, 链接器会尝试把 U 中的所有未解析符号与 f 中各目标模块定义的符号进行匹配。如果某个目标模块 m 定义了一个 U 中的未解析符号 x , 那么就把 m 加入到 E 中, 并把符号 x 从 U 移入 D 中。不断地对 f 中的所有目标模块重复这个过程直到 U 和 D 不再变化为止。那些未加入到 E 中的 f 里的目标模块就被简单地丢弃, 链接器继续处理下一输入文件。

③ 如果处理过程中往 D 加入一个已存在的符号或者当扫描完所有输入文件时 U 非空, 链接器报错并停止动作。否则, 它把 E 中的所有目标文件进行重定位后合并在一起, 以生成可执行目标文件。

4.3.3 与静态库的链接

在类 UNIX 系统中, 静态库文件采用一种称为存档档案 (archive) 的特殊文件格式, 使用 .a 后缀。例如, 标准 C 函数库文件名为 libc.a, 其中包含一组广泛使用的标准 I/O 函数、字符串处理函数和整数处理函数, 如 atoi、printf、scanf、strcpy 等, libc.a 是默认的用于静态链接的库文件, 无需在链接命令中显式指出。还有其他的函数库, 例如浮点数运算函数库, 文件名为 libm.a, 其中包含 sin、cos 和 sqrt 函数等。

用户也可以自定义一个静态库文件。以下通过一个简单例子来说明如何生成自己的静态库文件。

假定有两个源文件 myproc1.c 和 myproc2.c, 如图 4.13 所示。

```
# include <stdio.h>
void myfunc1()
{
    printf("%s", "This is myfunc1 from mylib!\n");
}
```

a) myproc1.c文件

```
# include <stdio.h>
void myfunc2()
{
    printf("%s", "This is myfunc2 from mylib!\n");
}
```

b) myproc2.c文件

图 4.13 静态库 mylib 中包含的函数的源文件

可以使用 AR 工具生成静态库, 在此之前需要用 “gcc -c” 命令将静态库中包含的目标模块先生成可重定位目标文件。以下两个命令可以生成静态库文件 mylib.a, 其中包含两个目标模块 myproc1.o 和 myproc2.o。

```
gcc -c myproc1.c myproc2.c
ar rcs mylib.a myproc1.o myproc2.o
```

假定有一个 main.c 程序, 其中调用了静态库 mylib.a 中的函数 myfunc1。

```
1 void myfunc1(void);
2 int main()
3 {
4     myfunc1();
5     return 0;
6 }
```

为了生成可执行文件 myproc, 可以先将 main.c 编译并汇编为可重定位目标文件 main.o, 然后再将 main.o 和 mylib.a 以及标准 C 函数库 libc.a 进行链接。以下两条命令可以完成上述功能。

```
gcc -c main.c
gcc -static -o myproc main.o ./mylib.a
```

命令中使用 -static 选项指示链接器应生成一个完全链接的可执行目标文件, 即生成的可执行文件应能直接加载到存储器执行, 而不需要在加载或运行时再动态链接其他目标模块。

命令“`gcc -static -o myproc main.o ./mylib.a`”中符号解析过程如下。

一开始 E 、 U 、 D 都是空集，链接器首先扫描到 `main.o`，把它加入 E ，同时把其中未解析符号 `myfun1` 加入 U ，把定义符号 `main` 加入 D ，而且因为 `main.o` 的默认静态链接库是 `libc.a`，所以 `libc.a` 被加入到当前输入文件列表的末尾。

处理完 `main.o`，接着扫描到 `mylib.a`，因为这是个静态库文件，所以会拿当前 U 中的所有符号（本例中就一个符号 `myfun1`）与 `mylib.a` 中的所有目标模块（本例中有两个目标模块 `myproc1.o` 和 `myproc2.o`）依次匹配，看是否有哪个模块定义了 U 中的符号，结果发现在 `myproc1.o` 中定义了 `myfun1`，于是 `myproc1.o` 被加入到 E ，`myfun1` 从 U 转移到 D 。在 `myproc1.o` 中发现还有未解析符号 `printf`，因而将其加到 U 中。同样，`mylib.a` 指定的默认标准库还是 `libc.a`，它已经被加到当前输入文件列表的末尾，因此在此可以忽略它。不断地在静态库 `mylib.a` 的各模块上进行迭代以匹配 U 中的符号，直到 U 、 D 都不再变化。显然，此时 U 和 D 就不再发生变化， U 中只有一个未解析符号 `printf`，而 D 中有 `main` 和 `myfun1` 两个定义符号。因为模块 `myproc2.o` 没有被加入 E 中，因而它被丢弃。

接着扫描下一个输入文件，就是默认的库文件 `libc.a`。链接器发现 `libc.a` 中的目标模块 `printf.o` 定义了符号 `printf`，于是 `printf` 也从 U 移到 D ，同时 `printf.o` 被加入到 E ，并把它定义的所有符号都加入 D ，而所有未解析符号加入 U 。链接器还会把每个程序都要用到的一些初始化操作所在的目标模块（如 `crt0.o` 等）以及它们所引用的模块（如 `malloc.o`、`free.o` 等）自动加入到 E 中，并更新 U 和 D 以反映这个变化。事实上，标准库中各目标模块里的未解析符号都可以在标准库内其他模块中找到定义，因此当链接器处理完 `libc.a` 时， U 一定是空的。此时，链接器合并 E 中的目标模块并输出可执行目标文件。

图 4.14 概括了上面描述的链接器中符号解析的全过程。

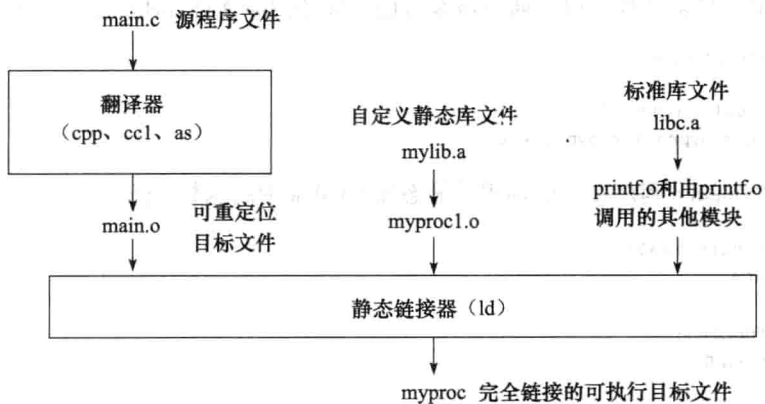


图 4.14 可重定位目标文件与静态库的链接

从上述描述的符号解析过程来看，符号解析结果与命令行中指定的输入文件的顺序相关。如果上述链接命令改为以下形式，则会发生链接错误。

```
gcc -static -o myproc ./mylib.a main.o
```

因为一开始先扫描到 `mylib.a`，而 `mylib.a` 为静态库文件，因此，会根据其中是否存在 U 中

的未解析符号对应的定义符号来确定是否将相应的目标模块加入 E 中, 显然, 开始时 U 是空的, 因而在 `mylib.a` 中没有任何一个目标模块被加入 E 中, 当扫描到 `main.o` 时, 其引用符号 `myfunc1` 便不能被解析, 所以被加入 U 中, 这样, U 中的 `myfunc1` 在后面将一直无法得到解析, 最终因为 U 不空而导致链接器输出错误信息并终止。

关于静态库的链接顺序问题, 通常的准则是将静态库文件放在命令行文件列表的后面, 如果有多个静态库文件, 则根据这些静态库文件的目标模块中的符号是否有引用关系来确定顺序。若相互之间都没有引用关系, 则说明它们之间相互独立, 此时顺序可以任意, 只要都放在后面即可; 若相互之间有引用关系, 则必须按照引用关系在命令行中排列静态库文件, 使得对于每个静态库目标模块中的外部引用符号, 在命令行中至少有一个包含其定义的静态库文件排在后面。例如, 假设 `func.o` 调用了静态库 `libx.a` 和 `liby.a` 中的函数, 而 `libx.a` 又调用了 `libz.a` 中的函数, 且 `libx.a` 和 `liby.a` 之间、`liby.a` 和 `libz.a` 之间是相互独立的, 则命令行中 `libx.a` 必须在 `libz.a` 之前, 而 `libx.a` 和 `liby.a` 之间、`liby.a` 和 `libz.a` 之间无需考虑顺序关系, 即以下几个命令行都是可行的。

```
gcc -static -o myfunc func.o libx.a liby.a libz.a
gcc -static -o myfunc func.o liby.a libx.a libz.a
gcc -static -o myfunc func.o libx.a libz.a liby.a
```

如果两个静态库的目标模块有相互引用关系, 则在命令行中可以重复静态库文件名。例如, 假设 `func.o` 调用了静态库 `libx.a` 中的函数, 而 `libx.a` 又调用了 `liby.a` 中的函数, 同时, `liby.a` 也调用了 `libx.a` 中的函数, 则可用以下命令进行链接。

```
gcc -static -o myfunc func.o libx.a liby.a libx.a
```

4.4 重定位

重定位的目的是在符号解析的基础上将所有关联的目标模块 (即上述集合 E 中的模块) 合并, 并确定运行时每个定义符号在虚拟地址空间中的地址, 在定义符号的引用处重定位引用的地址。例如, 对于图 4.14 中的例子, 编译 `main.c` 时, 因为编译器还不知道函数 `myproc1` 的地址, 所以编译器只是将一个“临时地址”放到可重定位目标文件 `main.o` 的 `call` 指令中, 在链接阶段, 这个“临时地址”将被修正为正确的引用地址, 这个过程叫重定位。具体来说, 重定位有以下两方面工作。

① 节和定义符号的重定位。链接器将所有模块中相同类型的节合并, 生成一个同一类型的新节。例如, 所有模块中的 `.data` 节合并为一个大的 `.data` 节, 它就是生成的可执行目标文件中的 `.data` 节。然后链接器根据每个新节在虚拟地址空间中的起始位置以及新节中每个定义符号的位置, 为新节中的每个定义符号确定存储地址。

② 引用符号的重定位。链接器对合并后新代码节 (`.text`) 和新数据节 (`.data`) 中的引用符号进行重定位, 使其指向对应的定义符号起始处。为了实现这一步工作, 显然, 链接器要知道目标文件中哪些引用符号需要重定位、所引用的是哪个定义符号等, 这些称为重定位信息, 放在重定位节 (`.rel.text` 和 `.rel.data`) 中。

4.4.1 重定位信息

在可重定位目标模块的 .rel.text 节和 .rel.data 节中，存放着每个需重定位的符号的重定位信息。 .rel.text 节和 .rel.data 节采用的数据类型是结构数组，每个数组元素是一个表项，每个表项对应一个需重定位的符号，表项的数据结构如下：

```
typedef struct {
    Elf32_Addr  r_offset;
    Elf32_Word  r_info;
} Elf32_Rel;
```

字段 r_offset 指出当前需重定位的位置相对于所在节起始位置的字节偏移量。若重定位的是变量的位置，则所在节为 .data 节；若重定位的是函数的位置，则所在节是 .text 节。r_info 指出当前需重定位的符号所引用的符号在符号表中的索引值以及相应的重定位类型。从以下的宏定义中可以看出，符号索引（用字段名 r_sym 表示）是 r_info 的高 24 位，重定位类型（r_type）是其低 8 位。

```
#define ELF32_R_SYM(info)      ((info)>>8)
#define ELF32_R_TYPE(info)    ((unsigned char)(info))
#define ELF32_R_INFO(sym, type) (((sym)<<8)+(unsigned char)(type))
```

重定位类型与特定的处理器有关。IA-32 处理器的重定位类型有 16 种之多，最基本的是以下两种。

① R_386_PC32：指明引用处采用相对寻址方式，即有效地址为 PC 内容加上重定位后的 32 位地址，PC 的内容是下条指令地址。例如，调用指令 call 中的转移目标地址就采用相对寻址方式。

② R_386_32：指明引用处采用绝对地址方式，即有效地址就是重定位后的 32 位地址。

重定位表的信息可以用命令“readelf -r”来显示，例如，用命令 readelf -r main.o 来显示 main.o 中的重定位表项。为方便起见，以下叙述中把重定位后的 32 位地址简称为重定位值。

4.4.2 重定位过程

重定位过程需要对 .text 节和 .data 节中由相应的重定位节 .rel.text 和 .rel.data 的重定位表项指出的每一处进行迭代执行。

对于图 4.7 所示例子，main.o 的 .rel.text 节中有一个表项：r_offset = 0x7，r_sym = 10，r_type = R_386_PC32，因此，需要在其 .text 节中偏移量为 0x7 的地方按照 PC 相对地址方式进行重定位，所引用的符号为 main.o 符号表中第 10 个表项代表的符号，根据图 4.8 可知，该符号为 swap。

对于图 4.7 所示例子，swap.o 的 .rel.data 中有一个表项：r_offset = 0x0，r_sym = 9，r_type = R_386_32，因此，需要在其 .data 节中偏移量为 0 的地方按绝对地址方式进行重定位，所引用的符号为 swap.o 符号表中第 9 个表项代表的符号，根据图 4.9 可知，该符号为 buf。

1. R_386_PC32 方式的重定位

对于图 4.7 所示例子，目标模块 main.o 中的 .text 节中主要是 main 函数的机器代码，其中

有一处需要重定位，就是与 main.c 中第 7 行 swap 函数对应的调用指令中的目标地址。

图 4.15 给出了 main.o 中 .text 节和 .rel.text 节的内容通过 OBJDUMP 工具反汇编出来的结果。

1	Disassembly of section .text:		
2	00000000 <main>:		
3	0: 55	push	%ebp
4	1: 89 e5	mov	%esp,%ebp
5	3: 83 e4 f0	and	\$0xffffffff0,%esp
6	6: e8 fc ff ff ff	call	7 <main+0x7>
7	7: R_386_PC32 swap		
8	b: b8 00 00 00 00	mov	\$0x0,%eax
9	10: c9	leave	
10	11: c3	ret	

图 4.15 main.o 中 .text 节和 .rel.text 节内容

从图 4.15 可以看出，符号 main 的定义从 .text 节中偏移量为 0 处开始，共占 18(0x12) 个字节。.rel.text 节中有一个重定位表项：r_offset = 0x7, r_sym = 10, r_type = R_386_PC32，被 OBJDUMP 工具以“7: R_386_PC32 swap”的可重定位信息显示在需重定位的 call 指令的下一行。call 指令中需重定位的是离 .text 节头偏移量为 0x7 的 4 字节地址，采用 PC 相对地址方式，重定位后应指向符号 swap 的定义处首地址。

假定重定位后在可执行文件中 main 函数对应的机器代码从 0x8048380 开始，紧跟在 main 后的是 swap 函数的机器代码，且首地址按 4 字节边界对齐，则 swap 的机器代码将从 0x8048394 开始，即符号 swap 的定义处首地址为 0x8048394(因为 0x8048380 + 0x12 = 0x8048392，要求 4 字节对齐的情况下就是 0x8048394)。

IA-32 中转移目标地址(即有效地址)计算公式为：转移目标地址 = PC + 偏移地址。这里 PC 是下条指令地址。call 指令中的重定位值就是偏移地址，因此重定位值 = 转移目标地址 - PC。这里的转移目标地址为符号 swap 的定义处首地址 0x8048394，PC 内容为 0x8048380 + 0x7 + 4 = 0x804838b，所以，重定位值应为 0x8048394 - 0x804838b = 0x9。因此，在可执行文件的 .text 节中，main 函数机器代码中 call 指令的代码修改为“e8 09 00 00 00”。

根据图 4.15 中 call 指令的机器代码“e8 fc ff ff ff”可知，需重定位的 4 字节地址的初始值(init)为 0xffffffff(注意，IA-32 为小端方式)，即 -4。汇编器用 -4 作为偏移量，其原因是，在 call 指令的执行过程中，需要进行转移目标地址计算，此时，PC 指向的是 call 指令的下条指令开始处，此处相对于需重定位的地址偏移为 4 个字节。

从上面分析过程可以看出，PC 相对地址方式下的重定位值计算公式为：

$$\text{ADDR}(r_sym) - ((\text{ADDR}(.text) + r_offset) - \text{init})$$

其中 ADDR(r_sym) 表示符号 r_sym 在运行时的存储地址。ADDR(.text) 表示节 .text 在运行时的起始存储地址，它加上偏移量 r_offset 后得到需重定位处的地址，再减初始值 init(相当于加 4)后，便得到 PC 值。ADDR(r_sym) 减 PC 值就是重定位值。例如，在上述例子中，ADDR(swap) = 0x8048394，ADDR(.text) = 0x8048380，r_offset = 0x7，init = -4。

2. R_386_32 方式的重定位

对于图 4.7 所示例子，因为 main.c 中只有一个已初始化的全局定义符号 buf，并且 buf 的定义没有引用其他符号，因此 main.o 中的 .data 节对应的重定位节 .rel.data 中没有任何重定位表项。main.o 中的 .data 节和 .rel.data 节的内容通过 OBJDUMP 工具反汇编出来的结果如图 4.16a 所示。

对于图 4.7 所示例子中的 swap.c，其中第 3 行有一个对全局变量 bufp0 赋初值的语句，bufp0 被初始化为外部数组变量 buf 的首地址。因而，在 swap.o 的 .data 节中有相应的对 bufp0 的定义，在 .rel.data 节中有对应的重定位表项。图 4.16b 给出了 swap.o 中 .data 节和 .rel.data 节的内容通过 OBJDUMP 工具反汇编出来的结果。

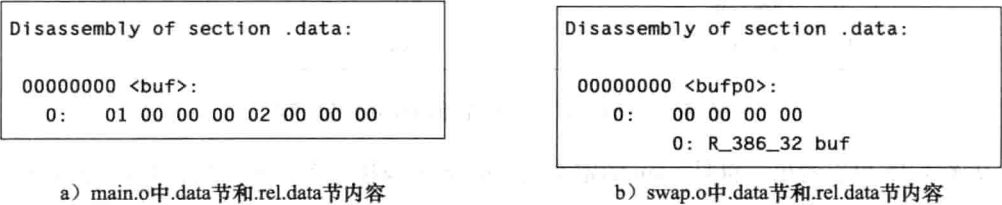


图 4.16 main.o 和 swap.o 中 .data 节和 .rel.data 节内容

从图 4.16b 可以看出，目标模块 swap 中全局符号 bufp0 的定义在 .data 节中偏移为 0 处开始，占 4 个字节，初始值（init）为 0x0。对应重定位节 .rel.data 中有一个重定位表项：r_offset = 0x0，r_sym = 9，r_type = R_386_32，OBJDUMP 工具解释后显示为“0: R_386_32 buf”。重定位类型是 R_386_32，即绝对地址方式，因而重定位值应是初始值加所引用符号地址。假定所引用符号 buf 在运行时的存储地址 ADDR(buf) = 0x8049620，则在可执行目标文件中重定位后的 bufp0 的内容变为 0x8049620，即“20 96 04 08”。

可执行目标文件中的 .data 节是将 main.o 中的 .data 节和 swap.o 中的 .data 节合并后生成的，经过重定位后得到合并后的 .data 节的内容，如图 4.17 所示。

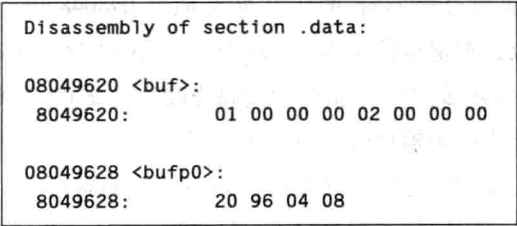


图 4.17 可执行目标文件中的 .data 节内容

可以看出，链接器进行重定位后，确定了运行时 .data 节在虚拟存储空间中的首地址为 0x8049620，这个地址就是 main.o 中定义的 buf 数组的第一个元素的地址，buf 有两个 int 型元素，因而占用了 8 个字节。从 swap.o 的 .data 节合并过来的 bufp0 从 0x8049628 开始，其内容为 buf 的首地址 0x8049620。

图 4.18 给出了 swap.o 中的 .text 节和 .rel.text 节的内容通过 OBJDUMP 工具反汇编出来的结果（大括弧部分是后加的功能说明）。

1	Disassembly of section .text:		
2	00000000 <swap>:		
3	0: 55	push	%ebp
4	1: 89 e5	mov	%esp,%ebp
5	3: 83 ec 10	sub	\$0x10,%esp
6	6: c7 05 00 00 00 00 04	movl	\$0x4,0x0
7	d: 00 00 00		
8	8: R_386_32	.bss	
9	c: R_386_32	buf	
10	10: a1 00 00 00 00	mov	0x0,%eax
11	11: R_386_32	bufp0	
12	15: 8b 00	mov	(%eax),%eax
13	17: 89 45 fc	mov	%eax,-0x4(%ebp)
14	1a: a1 00 00 00 00	mov	0x0,%eax
15	1b: R_386_32	bufp0	
16	1f: 8b 15 00 00 00 00	mov	0x0,%edx
17	21: R_386_32	.bss	
18	25: 8b 12	mov	(%edx),%edx
19	27: 89 10	mov	%edx,(%eax)
20	29: a1 00 00 00 00	mov	0x0,%eax
21	2a: R_386_32	.bss	
22	2e: 8b 55 fc	mov	-0x4(%ebp),%edx
23	31: 89 10	mov	%edx,(%eax)
24	33: c9	leave	
25	34: c3	ret	

图 4.18 swap.o 中的 .text 节和 .rel.text 节的内容

从图 4.18 可看出, 符号 swap 从 .text 节中偏移为 0 处开始, 占 52 字节。在对应的 .rel.text 节中有 6 个表项, 分别指出需要在第 0x8、0xc、0x11、0x1b、0x21 和 0x2a 处 (即指令中加粗部分) 进行重定位, 全部为绝对地址方式 (即 R_386_32), 分别引用符号 bufp1、buf、bufp0、bufp0、bufp1、bufp1 的存储地址。

由图 4.17 可知, buf 和 bufp0 的存储地址分别是 0x8049620 和 0x8049628, 而符号 bufp1 的地址就是链接合并后 .bss 节的首地址, 假定为 0x8049700, 则链接生成的可执行文件的 .text 节中的内容如下所示。

```

08048380 <main>:
08048380: 55                push    %ebp
08048381: 89 e5            mov     %esp,%ebp
08048383: 83 e4 f0        and     $0xfffffffff0,%esp
08048386: e8 09 00 00 00  call    8048394 <swap>
0804838b: b8 00 00 00 00  mov     $0x0,%eax
08048390: c9              leave   %eax
08048391: c3              ret
08048392: 90              nop
08048393: 90              nop

08048394 <swap>:
08048394: 55                push    %ebp
08048395: 89 e5            mov     %esp,%ebp
08048397: 83 ec 10        sub     $0x10,%esp
0804839a: c7 05 00 97 04 08 24 mov     $0x8049624,0x8049700
080483a1: 96 04 08

```

```
80483a4: a1 28 96 04 08      mov     0x8049628,%eax
80483a9: 8b 00                mov     (%eax),%eax
80483ab: 89 45 fc              mov     %eax,-0x4(%ebp)
80483ae: a1 28 96 04 08      mov     0x8049628,%eax
80483b3: 8b 15 00 97 04 08    mov     0x8049700,%edx
80493b9: 8b 12                mov     (%edx),%edx
80493bb: 89 10                mov     %edx,(%eax)
80493bd: a1 00 97 04 08      mov     0x8049700,%eax
80493c2: 8b 55 fc              mov     -0x4(%ebp),%edx
80493c5: 89 10                mov     %edx,(%eax)
80493c7: c9                  leave
80493c8: c3                  ret
```

上述可执行目标文件中的 .text 节是由 main.o 和 swap.o 两个目标模块中的 .text 节合并而来的，在可执行目标文件的 .text 节中真正存储的信息只是中间的机器代码，左边的地址和右边的汇编指令都是 OBJDUMP 工具根据图 4.5 所示的可执行目标文件中的程序头表信息和指令代码本身反汇编出来的。合并过程如图 4.19 所示。从图 4.19 可以看出，在可执行目标文件的 .text 节和 .data 节中还分别包含系统代码（system code）和系统数据（system data）。

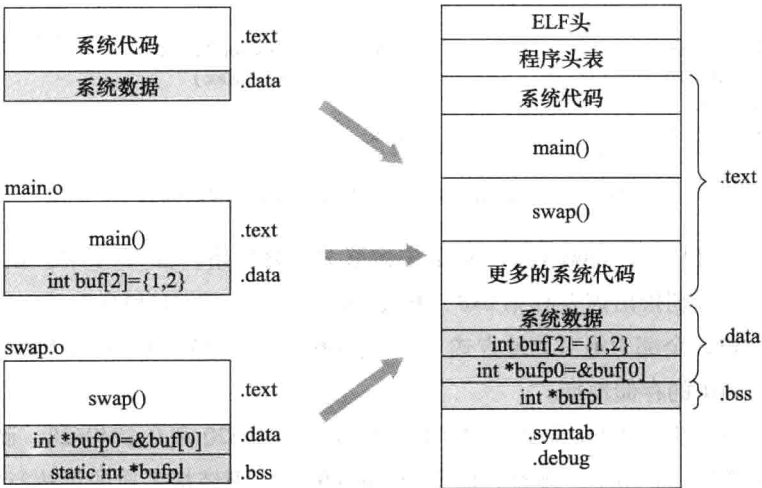


图 4.19 main.o 和 swap.o 合并成可执行文件

4.5 可执行文件的加载

当启动一个可执行目标文件执行时，首先会通过某种方式调出常驻内存的一个称为加载器（loader）的操作系统程序来进行处理。例如，任何 UNIX 程序的加载执行都是通过调用 execve 系统调用函数来启动加载器进行的。加载器根据可执行目标文件中的程序头表信息，将可执行目标文件中相关节的内容与虚拟地址空间中的只读代码段和可读写数据段通过页表建立映射，然后启动可执行目标文件中的第一条指令执行。

对于特定的系统平台，每个可执行目标文件在运行时的存储器映像是确定的。如图 4.20 所示是 Linux 系统中可执行目标文件的存储器映像。

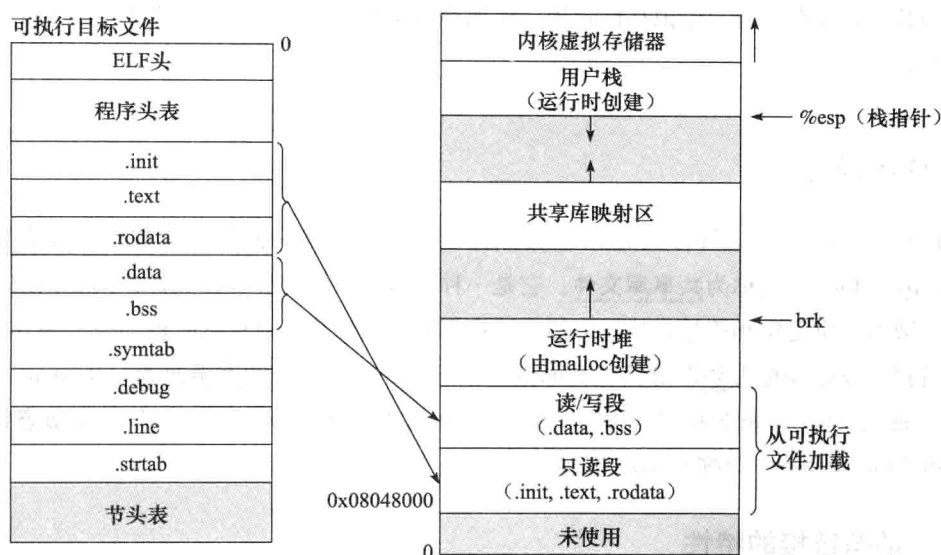


图 4.20 Linux 下可执行目标文件运行时的存储器映像

将图 4.20 与第 6 章的图 6.30 比较，可以看出这里的存储器映像是指可执行目标文件中的代码段和数据段在虚拟地址空间中的映射关系。在图 4.6 所示的可执行目标文件的程序头表中，记录了可执行目标文件中所有存储段的相关信息，如存储段类型、段在虚拟地址空间中的起始地址、长度、对齐方式、访问权限等。其中，只读代码段（简称为只读段）和可读写数据段（简称为读/写段）都是可装入段（PT_LOAD），从图 4.20 可看出，只读段（包括 .init 节、.text 节和 .rodata 节）总是映射到虚拟地址为 0x08048000 开始的一段区域，可读写数据段（包括 .data 节和 .bss 节）则映射到只读段后面按 4KB 对齐的高地址上，其中 .bss 节所在存储区在运行时被初始化为 0。运行时堆（run-time heap）则在可读写数据段后面 4KB 对齐的高地址处，通过调用 malloc 库函数动态向高地址分配空间，而运行时的用户栈（user stack）则是从用户空间的最高地址往低地址方向增长。堆区和栈区中间有一块空间保留给共享库目标代码，栈区以上的高地址区是操作系统内核的虚拟存储区。

每个可执行目标文件都采用如图 4.20 所示的存储器映像方式，映射到一个统一的虚拟地址空间上，使得链接器在重定位时可以完全按照一个统一的虚拟存储空间来确定每个符号的地址，而不用管其数据和代码将来存放在主存或磁盘的何处。因此，引入虚拟存储管理简化了链接器的设计和实现。

同样，引入虚拟存储管理也简化了程序加载过程。因为统一的虚拟地址空间映像使得每个可执行目标文件的只读代码段都映射到 0x08048000 开始的一块连续区域，可读写数据段映射到虚拟地址空间中的一块连续区域，因而加载器可以非常容易地对这些连续区域进行分页，并初始化相应页表项的内容。加载时，只读代码段和可读写数据段对应的页表项都被初始化为“未缓存页”（即有效位为 0），并指向磁盘中可执行目标文件中适当的地方。因此，程序加载过程中，实际上并没有真正从磁盘上加载代码和数据到主存，而是仅仅创建了只读代码段和可读写数据段对应的页表项。只有在执行代码过程中发生了“缺页”异常，才会真正从磁盘加

载代码和数据到主存。有关虚拟存储管理、虚拟地址空间、页表、缺页异常等相关内容见第6章的6.5节。

4.6 动态链接

前面介绍了可重定位目标文件和可执行目标文件，还有一类目标文件是共享目标文件 (shared object file)，也称为共享库文件。它是一种特殊的可重定位目标文件，其中记录了相应的代码、数据、重定位和符号表信息，能在可执行目标文件装入或运行时被动态地装入内存并自动被链接，该过程称为动态链接 (dynamic link)，由一个称为动态链接器 (dynamic linker) 的程序来完成。UNIX 系统中共享库文件采用 .so 后缀，Windows 系统中称其为动态链接库 (Dynamic Link Library，简称 DLL)，采用 .dll 后缀。

4.6.1 动态链接的特性

共享库以动态链接的方式被多个加载过程中或正在执行的应用程序共享，因而共享库的动态链接有两方面的特点：一是“共享性”，二是“动态性”。

“共享性”是指共享库中的代码段和数据段在内存只有一个副本，当应用程序在其代码中需要引用共享库中的符号时，在引用处通过某种方式确定指向共享库中对应定义符号的地址即可。例如，对于动态共享库 libc.so 中的 printf 模块，内存中只有一个 printf 副本，所有应用程序都可以通过动态地链接 printf 模块来使用它。而在静态库链接方式下，静态库 libc.a 中的 printf 模块会在静态链接时被合并到每个可执行文件中，因而是嵌入到应用程序中的，在多个使用 printf 的应用程序同时在系统中执行时，这些程序中的 printf 代码都会占用内存空间，也即在内存中有多个 printf 副本。因此，静态库链接方式对于并发运行几十个进程的系统来说，会造成极大的主存资源浪费，而动态链接的共享库方式，则能节省主存资源。

“动态性”是指共享库只在使用它的程序被加载或执行时才加载到内存，因而在共享库更新后并不需要重新对程序进行链接，每次加载或执行程序时所链接的共享库总是最新的。可以利用共享库的这个特性来实现软件分发或生成动态 Web 网页等。而静态库则需要程序员定期维护和更新，关注它是否有新版本出现，在出现新版本时需要重新对程序进行链接操作。

动态链接有两种方式，一种是在程序加载过程中加载和链接共享库，另一种是在程序执行过程中加载和链接共享库。

4.6.2 程序加载时的动态链接

在类 UNIX 系统中，共享库文件使用 .so 后缀。例如，标准 C 函数库文件名为 libc.so。用户也可以自定义一个动态共享库文件。例如，对于图 4.13 所示的两个源程序文件 myproc1.c 和 myproc2.c，可以使用以下 GCC 命令生成动态链接的共享库 mylib.so。

```
gcc -shared -fPIC -o mylib.so myproc1.c myproc2.c
```

上述命令中 -shared 选项告诉链接器生成一个共享库目标文件；-fPIC 选项告诉编译器生成

与位置无关的代码 (Position Independent Code, PIC), 使得共享库在被任何不同的程序引用时都不需要修改其代码。这保证了共享库代码的存储位置可以是不确定的, 而且即使共享库代码的长度发生改变也不会影响调用它的程序。

假定有一个 main.c 程序, 其中调用了 mylib.so 中的函数 myfunc1。

```
void myfunc1(void);
int main()
{
    myfunc1();
    return 0;
}
```

为了生成可执行目标文件 myproc, 可以先将 main.c 编译并汇编为可重定位目标文件 main.o, 然后再将 main.o 和 mylib.so 以及标准 C 函数共享库 libc.so 进行链接。以下命令可以完成上述功能:

```
gcc -o myproc main.c ./mylib.so
```

通过上述命令得到可执行目标文件 myproc, 这个命令与静态链接命令 “gcc -static -o myproc main.c mylib.a” 的执行过程不同。静态链接生成的可执行目标文件在加载后可以直接运行, 因为所有外部函数都已包含在可执行目标文件中。而动态链接生成的可执行目标文件不能运行, 这是因为在动态链接生成可执行目标文件时, 其中对外部函数的引用地址是未知的。因此, 在动态链接生成的可执行目标文件运行前, 系统会首先将动态链接器以及所使用的共享库文件加载到内存。动态链接器和共享库文件的路径都包含在可执行目标文件中, 其中, 动态链接器由加载器加载, 而共享库由动态链接器加载。

图 4.21 给出了动态链接的全过程。整个过程被分成两步: 首先, 进行静态链接以生成部分链接的可执行目标文件 myproc, 该文件中仅包含共享库 (包括指定的共享目标文件 mylib.so 和默认的标准共享库文件 libc.so) 中的符号表和重定位表信息, 而共享库中的代码和数据并没有被合并到 myproc 中; 然后, 在加载 myproc 时, 由加载器将控制权转移到指定的动态链接器, 由动态链接器对共享目标文件 libc.so、mylib.so 和 myproc 中的相应模块内的代码和数据进行重定位并加载共享库, 以生成最终的存储空间中完全链接的可执行目标, 在完成重定位和加载共享库后, 动态链接器把控制权转移到程序 myproc。在执行 myproc 的过程中, 共享库中的代码和数据在存储空间的位置一直是固定的。

在上述过程中, 有一个重要的问题是, 如何在加载过程中将控制权从加载器转移到动态链接器? 参看图 4.6 可以发现, 在可执行目标文件的程序头表中有

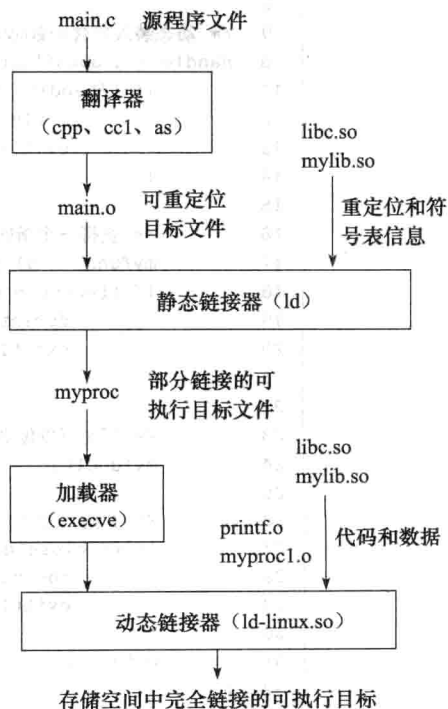


图 4.21 采用加载时动态链接的过程

一个 Type = INTERP 的段。因此，这个问题的解决可通过在可执行目标文件 myproc 中加入一个特殊的 .interp 节来实现。当加载 myproc 时，加载器会发现在 myproc 的程序头表中包含了 .interp 节构成的段，其 p_type 字段取值为 PT_INTERP，该节中包含了动态链接器的路径名，而动态链接器本身也是一个共享目标，在 Linux 系统中为 ld-linux.so，因而可以由加载器根据指定的路径来加载并启动动态链接器运行。动态链接器完成相应的重定位工作后再把控制权交给 myproc，启动其第一条指令执行。

4.6.3 程序运行时的动态链接

图 4.21 描述的是在程序被加载时对共享库进行动态链接的过程，实际上，共享库也可以在程序运行过程中进行动态链接。在一些类 UNIX 系统中，提供了一个动态链接器接口，其中定义了相应的几个函数，如 dlopen、dlsym、dlerror、dlclose 等，其头文件为 dlfcn.h。以下给出一个例子，用以说明如何在应用程序中使用动态链接器接口函数对共享库进行动态链接。

图 4.22 给出了一个运行时进行动态链接的应用程序示例 main.c。对于由图 4.13 所示的两个源程序文件 myproc1.c 和 myproc2.c 生成的共享库 mylib.so，在 main.c 中调用了共享库 mylib.so

```

1  #include <stdio.h>
2  #include <dlfcn.h>
3  int main()
4  {
5      void *handle;
6      void (*myfunc1)();
7      char *error;
8
9      /* 动态装入包含函数myfunc1()的共享库文件 */
10     handle = dlopen("./mylib.so", RTLD_LAZY);
11     if (!handle) {
12         fprintf(stderr, "%s\n", dlerror());
13         exit(1);
14     }
15
16     /* 获得一个指向函数myfunc1()的指针myfunc1*/
17     myfunc1 = dlsym(handle, "myfunc1");
18     if ((error = dlerror()) != NULL) {
19         fprintf(stderr, "%s\n", error);
20         exit(1);
21     }
22
23     /* 现在可以像调用其他函数一样调用函数myfunc1() */
24     myfunc1();
25
26     /* 关闭（卸载）共享库文件 */
27     if (dlclose(handle) < 0) {
28         fprintf(stderr, "%s\n", dlerror());
29         exit(1);
30     }
31     return 0;
32 }
```

图 4.22 采用运行时动态链接的应用程序 main.c

中的函数 `myfunc1`。要编译该程序并生成可执行文件 `myproc`，通常用以下 GCC 命令：

```
gcc -rdynamic -o myproc main.c -ldl
```

选项 `-rdynamic` 指示链接器在链接时使用共享库中的函数，选项 `-ldl` 说明采用动态链接器接口中的 `dlopen`、`dlsym` 等函数进行运行时的动态链接。

如图 4.22 中源程序 `main.c` 所示，一个应用程序如果要在运行时动态链接一个共享库并引用库中的函数或变量，则必须经过以下几个步骤。

① 首先，通过 `dlopen` 函数加载和链接共享库，如图 4.22 中第 10 行所示。第 10 行的含义是启动动态链接器来加载并链接当前目录中的共享库文件 `mylib.so`，这里 `dlopen` 函数的第二个参数为 `RTLD_LAZY`，用来指示链接器对共享库中外部符号的解析时间推迟到执行库中代码时。若 `dlopen` 函数出错，则返回值为 `NULL`；否则返回指向共享库文件句柄的指针。

② 在 `dlopen` 函数正常返回的情况下，通过 `dlsym` 函数获取共享库中所需函数。如图 4.22 中第 17 行所示。第 17 行的含义是指动态链接器返回指定共享库 `mylib.so` 中指定符号 `myfunc1` 的地址。若指定共享库中不存在指定的符号，则返回 `NULL`。`dlsym` 函数的第一个参数是指定共享库的文件句柄，第二个参数用来标识指定符号的字符串，通常是后面将要使用的函数的函数名。

③ 在 `dlsym` 函数正常返回的情况下，就可以使用共享库中的函数，如图 4.22 中第 24 行所示。函数对应代码的首地址由 `dlsym` 函数返回。

④ 在使用完程序所需的所有共享库内函数或变量后，使用 `dlclose` 函数卸载这个共享库。如图 4.22 中第 27 行所示。若卸载成功，返回为 0，否则为 -1。

若调用 `dlopen`、`dlsym` 和 `dlclose` 时发生出错，则出错信息可通过调用 `dLError` 函数获得。

4.7 小结

链接器位于编译器、指令集体系结构和操作系统的交叉点上，涉及指令系统、代码生成、机器语言、程序转换和虚拟存储管理等诸多概念，因而它对于理解整个计算机系统概念来说是非常重要的。

链接处理涉及三种目标文件格式：可重定位目标文件、可执行目标文件和共享目标文件。共享库文件是一种特殊的可重定位目标文件。ELF 目标文件格式有链接视图和执行视图两种，前者是可重定位目标格式，后者是可执行目标格式。链接视图中包含 ELF 头、各个节以及节头表；执行视图中包含 ELF 头、程序头表（段头表）以及各种节组成的段。

链接分为静态链接和动态链接两种，静态链接处理的是可重定位目标文件，它将多个可重定位目标模块中相同类型的节合并起来，以生成完全链接的可执行目标文件，其中所有符号的引用都是确定的在虚拟地址空间中的最终地址，因而可以直接被加载执行。而动态链接方式下的可执行目标文件是部分链接的，还有一部分符号的引用地址没有确定，需要利用共享库中定义的符号进行重定位，因而需要由动态链接器来加载共享库并重定位可执行文件中部分符号的引用。动态链接有两种方式，一种是可执行目标文件加载时进行共享库的动态链接；另一种是

可执行目标文件在执行时进行共享库的动态链接。

链接过程需要完成符号解析和重定位两方面的工作，符号解析的目的就是将符号的引用与符号的定义关联起来，重定位的目的是分别合并代码和数据，并根据代码和数据在虚拟地址空间中的位置，确定每个符号的最终存储地址，然后根据符号的确切地址来修改符号的引用处的地址。

在不同的目标模块中可能会定义相同的符号，因为相同的多个符号只能分配一个地址，所以链接器需要确定以哪个符号为准。编译器通过对定义符号标识其为强符号还是弱符号，由链接器根据一套规则来确定多重定义符号中哪个是唯一的定义符号，如果不了解这些规则，则可能无法理解程序执行的有些结果。

加载器在加载可执行目标文件时，实际上只是把可执行目标文件中的只读代码段和可读写数据段通过页表映射到了虚拟地址空间中确定的位置，并没有真正把代码和数据从磁盘装入主存。

习题

1. 给出以下概念的解释说明。

链接	可重定位目标文件	可执行目标文件	符号解析
重定位	ELF 目标文件格式	ELF 头	节头表
程序头表（段头表）	只读代码段	可读写数据段	全局符号
外部符号	本地符号	强符号	弱符号
多重定义符号	静态库	符号的定义	符号的引用
未解析符号	重定位信息	运行时堆	用户栈
动态链接	共享库（目标）文件		

2. 简单回答下列问题。

- (1) 如何将多个 C 语言源程序模块组合起来生成一个可执行目标文件？简述从源程序到可执行机器代码的转换过程？
- (2) 引入链接的好处是什么？
- (3) 可重定位目标文件和可执行目标文件的主要差别是什么？
- (4) 静态链接方式下，静态链接器主要完成哪两方面的工作？
- (5) 可重定位目标文件的 .text 节、.rodata 节、.data 节和 .bss 节中分别主要包含什么信息？
- (6) 可执行目标文件的 .text 节、.rodata 节、.data 节和 .bss 节中分别主要包含什么信息？
- (7) 可执行目标文件中有哪两种可装入段？哪些节组合成只读代码段？哪些节组合成可读写数据段？
- (8) 加载可执行目标文件时，加载器根据其中的哪个表的信息对可装入段进行映射？
- (9) 在可执行目标文件中，可装入段被映射到虚拟存储空间，这种做法有什么好处？
- (10) 静态链接和动态链接的差别是什么？

3. 假设一个 C 语言程序有两个源文件：main.c 和 test.c，它们的内容如图 4.23 所示。

<pre> 1 /* main.c */ 2 int sum(); 3 4 int a[4]={1, 2, 3, 4}; 5 extern int val; 6 int main() 7 { 8 val=sum(); 9 return val; 10 }</pre>	<pre> 1 /* test.c */ 2 extern int a[]; 3 int val=0; 4 int sum() 5 { 6 int i; 7 for (i=0; i<4; i++) 8 val += a[i]; 9 return val; 10 }</pre>
---	--

图 4.23 题3用图

对于编译生成的可重定位目标文件 test.o，填写下表中各符号的情况，说明每个符号是否出现在 test.o 的符号表 (.symtab 节) 中，如果是的话，定义该符号的模块是 main.o 还是 test.o，该符号的类型是全局、外部还是本地，该符号出现在 test.o 中的哪个节 (.text、.data 或 .bss)？

符号	是否在 test.o 的符号表中	定义模块	符号类型	节
a				
val				
sum				
i				

4. 假设一个 C 语言程序有两个源文件：main.c 和 swap.c，其中，main.c 的内容如图 4.7a 所示，而 swap.c 的内容如下：

```

1  extern int buf[];
2  int *bufp0 = &buf[0];
3  static int *bufp1;
4
5  static void incr() {
6      static int count=0;
7      count++;
8  }
9
10 void swap(){
11     int temp;
12     incr();
13     bufp1 = &bufp[1];
14     temp = *bufp0;
15     *bufp0 = *bufp1;
16     *bufp1 = temp;
17 }
```

对于编译生成的可重定位目标文件 swap.o，填写下表中各符号的情况，说明每个符号是否出现在 swap.o 的符号表 (.symtab 节) 中，如果是的话，定义该符号的模块是 main.o 还是 swap.o，该符号的类型是全局、外部还是本地，该符号出现在 swap.o 中的哪个节 (.text、.data 或 .bss)？

符号	是否在 swap.o 的符号表中	定义模块	符号类型	节
buf				
bufp0				
bufp1				
incr				
count				
swap				
temp				

5. 假设一个 C 语言程序有两个源文件：main.c 和 procl.c，它们的内容如图 4.24 所示。

```
1  #include <stdio.h>
2  unsigned x=257;
3  short y,z=2;
4  void procl(void);
5  void main()
6  {
7      procl();
8      printf("x=%u,z=%d\n", x, z);
9      return 0;
10 }
```

a) main.c 文件

```
1  double x;
2
3  void procl()
4  {
5      x=-1.5;
6  }
```

b) procl.c 文件

图 4.24 题 5 用图

回答下列问题。

- (1) 在上述两个文件中出现的符号哪些是强符号？哪些是弱符号？
 - (2) 程序执行后打印的结果是什么？请分别画出执行第 7 行的 procl() 函数调用前、后，在地址 &u 和 &s 中存放的内容。
 - (3) 修改文件 procl，使得 main.c 能输出正确的结果（即 x = 257，z = 2）。要求修改时不能改变任何变量的数据类型和名字。
6. 以下每一小题给出了两个源程序文件，它们被分别编译生成可重定位目标模块 m1.o 和 m2.o。在模块 mj 中对符号 x 的任意引用与模块 mi 中定义的符号 x 关联记为 REF(mj.x)→DEF(mi.x)。请在下列空格处填写模块名和符号名以说明给出的引用符号所关联的定义符号，若发生链接错误则说明其原因，若从多个定义符号中任选则给出全部可能的定义符号，若是局部变量则说明不存在关联。

```
(1) /* m1.c */
int pl(viod);
int main()
{
    int pl = pl();
    return pl;
}
```

```
/* m2.c */
static int main=1;
int pl()
{
    main++;
    return main;
}
```

① REF(m1.main)→DEF(_____)

② REF(m2.main)→DEF(_____._____)

③ REF(m1.p1)→DEF(_____._____)

④ REF(m2.p1)→DEF(_____._____)

```
(2) /* m1.c */           /* m2.c */
      int x=100;           float x=100.0;
      int p1(viod);        int main=1;
      int main()            int p1()
      {                     {
          x=p1();            main++;
          return x;          return main;
      }                     }
```

① REF(m1.main)→DEF(_____._____)

② REF(m2.main)→DEF(_____._____)

③ REF(m1.x)→DEF(_____._____)

```
(3) /* m1.c */           /* m2.c */
      int p1(viod);        int x=10;
      int p1;              int main;
      int main()            int p1()
      {                     {
          int x=p1();        main=1;
          return x;          return x;
      }                     }
```

① REF(m1.main)→DEF(_____._____)

② REF(m2.main)→DEF(_____._____)

③ REF(m1.p1)→DEF(_____._____)

④ REF(m1.x)→DEF(_____._____)

⑤ REF(m2.x)→DEF(_____._____)

```
(4) /* m1.c */           /* m2.c */
      int p1(viod);        double x=10;
      int x, y;             int y;
      int main()            int p1()
      {                     {
          x=p1();            y=1;
          return x;          return y;
      }                     }
```

① REF(m1.x)→DEF(_____._____)

② REF(m2.x)→DEF(_____._____)

③ REF(m1.y)→DEF(_____._____)

④ REF(m2.y)→DEF(_____._____)

7. 以下由两个目标模块 m1 和 m2 组成的程序，经编译、链接后在计算机上执行，结果发现即使 p1 中没有对数组变量 main 进行初始化，最终也能打印出字符串“0x5589\n”。为什么？要求解释原因。

```

1  /* m1.c */
2  void p1(viod);
3
4  int main()
5  {
6      p1();
7      return 0;
8  }

1  /* m2.c */
2  #include <stdio.h>;
3  char main[2];
4
5  void p1()
6  {
7      printf("0x%x%x\n", main[0], main[1]);
8  }

```

8. 图 4.25 中给出了用 OBJDUMP 显示的某个可执行目标文件的程序头表（段头表）的部分信息，其中，可读写数据段（Read/write data segment）的信息表明，该数据段对应虚拟存储空间中起始地址为 0x8049448、长度为 0x104 个字节的存储区，其数据来自可执行文件中偏移地址 0x448 开始的 0xe8 个字节。这里，可执行目标文件中的数据长度和虚拟地址空间中的存储区大小之间相差了 28 字节。请解释可能的原因。

```

Read-only code segment
LOAD off 0x00000000 vaddr 0x08048000 paddr 0x08048000 align 2**12
      filesz 0x00000448 memsz 0x00000448 flags r-x

Read/write data segment
LOAD off 0x00000448 vaddr 0x08049448 paddr 0x08049448 align 2**12
      filesz 0x000000e8 memsz 0x00000104 flags rw-

```

图 4.25 某可执行目标文件程序头表的部分内容

9. 假定 a 和 b 是可重定位目标文件或静态库文件，a→b 表示 b 中定义了一个被 a 引用的符号。对于以下每一小标题出现的情况，给出一个最短命令行（含有最少数量的可重定位目标文件或静态库文件参数），使得链接器能够解析所有的符号引用。
- (1) p.o→libx.a→liby.a→p.o
 - (2) p.o→libx.a→liby.a 同时 liby.a→libx.a
 - (3) p.o→libx.a→liby.a→libz.a 同时 liby.a→libx.a→libz.a
10. 图 4.15 给出了图 4.7a 所示的 main 源代码对应的 main.o 文件中 .text 节和 .rel.text 节的内容，图中显示其 .text 节中有一处需重定位。假定链接后 main 函数代码起始地址是 0x8048386，紧跟在 main 后的是 swap 函数的代码，且首地址按 4 字节边界对齐。要求根据对图 4.15 的分析，指出 main.o 的 .text 节中需重定位的符号名、相对于 .text 节起始位置的位移、所在指令行号、重定位类型、重定位前的内容、重定位后的内容，并给出重定位值的计算过程。
11. 图 4.18 给出了图 4.7b 所示的 swap 源代码对应的 swap.o 文件中 .text 节和 .rel.text 节的内容，图中显示 .text 节中共有 6 处需重定位。假定链接后生成的可执行目标文件中 buf 和 bufp0 的存储地址分别是 0x80495c8 和 0x80495d0，bufp1 的存储地址位于 .bss 节的开始，为

0x8049620。根据对图 4.18 的分析，仿照例子填写下表，指出各个重定位的符号名、相对于 .text 节起始位置的位移、所在指令行号、重定位类型、重定位前的内容、重定位后的内容。

序号	符号	位移	指令所在行号	重定位类型	重定位前内容	重定位后内容
1	bufp1 (.bss)	0x8	6 ~ 7	R_386_32	0x00000000	0x8049620
2						
3						
4						
5						
6						

可执行目标文件的运行

- 第5章 程序的执行
- 第6章 层次结构存储系统
- 第7章 异常控制流
- 第8章 I/O 操作的实现